SQLite with Python – Step-by-Step Guide

Using VS Code + SQLite Viewer Extension

WHAT YOU NEED BEFORE YOU START

Software required

You must have:

- **Python 3.10 or later**
- **VS Code**
- **SQLite Viewer extension for VS Code** (viewing only, not editing)

INSTALL SQLITE VIEWER EXTENSION (DO THIS ONCE)

1. Open **VS Code**
2. Click **Extensions** (left sidebar) or press: `Ctrl + Shift + X`
3. Search for:
4. `SQLite Viewer`
5. Install **SQLite Viewer** (Publisher: Florian Klampfer)

This extension is for **viewing only**.
All database changes must come from **Python code**, not the viewer.

STEP 1 – CREATE YOUR PROJECT FOLDER

1. Create a new folder on your computer 2.

Name the folder exactly: `sql_python_backend`

3. Open **VS Code**
4. Click **File → Open Folder**
5. Select `sql_python_backend`

STEP 2 – CREATE YOUR PYTHON FILE

1. In VS Code, open the **Explorer** panel
2. Right-click inside the folder
3. Click **New File**
4. Name the file exactly: `database_app.py`

This file will:

- connect to the database
- send SQL commands
- manage all data operations

STEP 3 – CREATE AND CONNECT TO THE DATABASE

Where to write this

At the **top** of `database_app.py` **What**

to write

```
import sqlite3

connection = sqlite3.connect("student_portal.db") cursor
= connection.cursor()
```

What this does

- Creates a database file called `student_portal.db`
- The file appears automatically in the folder
- No manual database creation is required

STEP 4 – CREATE DATABASE TABLES

Where to write this

Below the connection code in `database_app.py` **What**

to write

```
cursor.execute("""
CREATE TABLE IF NOT EXISTS users (
id INTEGER PRIMARY KEY,     username
TEXT NOT NULL UNIQUE,     email TEXT
NOT NULL UNIQUE
)
""")
cursor.execute("""
CREATE TABLE IF NOT EXISTS records (
id INTEGER PRIMARY KEY,     user_id
INTEGER,     score INTEGER NOT NULL,
```

```
    FOREIGN KEY (user_id) REFERENCES users(id)
)
""")
connection.commit()
```

What to do next

1. Save the file
2. Run the file:
   - Click **Run ▶** in VS Code
     OR
   - Open terminal in VS Code and run:
   - `python database_app.py`

What should happen

- No errors
- Database and tables created silently

STEP 5 – VIEW THE DATABASE USING SQLITE VIEWER (IMPORTANT)

This replaces DB Browser completely

1. In VS Code **Explorer**, locate:
2. `student_portal.db`
3. **Right-click** on `student_portal.db`
4. Click **"Open Database"**

What you will see

- A new SQLite Viewer tab opens
- Tables list appears (`users`, `records`) • You can:
  - View table structure ○ View rows ○ Confirm inserts worked

DO NOT:

- Edit data
- Run SQL manually

- Change table structure

All changes must come from Python.

STEP 6 – INSERT DATA INTO TABLES

Add this code to `database_app.py`

```
cursor.execute(
    "INSERT INTO users (username, email) VALUES (?, ?)",
    ("alice", "alice@email.com")
)
cursor.execute(
    "INSERT INTO users (username, email) VALUES (?, ?)",
    ("bob", "bob@email.com")
)
connection.commit()
```

Run the file again

```
python database_app.py
```

Check using SQLite Viewer

- Right-click `student_portal.db`
- Open Database
- Click **users** table
- Confirm two rows exist

STEP 7 – INSERT RELATED DATA (RECORDS)

Add this code

```
cursor.execute(
    "INSERT INTO records (user_id, score) VALUES (?, ?)",
(1, 85)
)
cursor.execute(
    "INSERT INTO records (user_id, score) VALUES (?, ?)",
(2, 90)
)
connection.commit()
```

Run the file again and refresh SQLite Viewer.

STEP 8 – READ DATA (SELECT)

Add this code

```
cursor.execute("SELECT * FROM users") users
= cursor.fetchall()
 for user in
users:
print(user)
```

Expected output in terminal

```
(1, 'alice', 'alice@email.com')
(2, 'bob', 'bob@email.com')
```

STEP 9 – FILTER DATA (WHERE)

Add this code

```
cursor.execute(
    "SELECT * FROM users WHERE username = ?",
    ("alice",)
)
user = cursor.fetchone() print(user)
```

STEP 10 – UPDATE DATA

Add this code

```
cursor.execute(
    "UPDATE users SET email = ? WHERE username = ?",
    ("alice_new@email.com", "alice")
)
connection.commit()
```

Run the file and refresh SQLite Viewer to confirm the update.

STEP 11 – DELETE DATA

Add this code

```
cursor.execute(
    "DELETE FROM records WHERE user_id = ?",
    (2,)
```

```
)
connection.commit()
```

Explain clearly to students:

DELETE removes data permanently.

STEP 12 – JOIN TABLES (IMPORTANT)

Add this code

```
cursor.execute("""
SELECT users.username, records.score
FROM users
JOIN records ON users.id = records.user_id
""")
results = cursor.fetchall()
 for row in
results:
print(row)
```

Example output

```
('alice', 85)
```

STEP 13 – WHY THIS IS SAFE BACKEND CODE

Explain:

- SQL uses placeholders (`?`)
- User input never touches raw SQL
- Backend controls all database actions
- Database enforces rules (UNIQUE, NOT NULL)

STEP 14 – CLOSE THE DATABASE CONNECTION

Always end your file with: `connection.close()`

FINAL FILE STRUCTURE (CHECK THIS)

Your folder should now contain:

```
sql_python_backend/ │
├── database_app.py
└── student_portal.db
```